EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Week 10-BN: Binary Ninja Personal — Hack Static & Dynamic Conditionals with the SG90 Servo (Raw `.bin`)

ARM Cortex-M33 • Binary Ninja • OpenOCD

October 12, 2026

For educational and defensive security research only

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

What You'll Learn This Week

- Build both lesson projects with `Release` and get an `.elf` and a raw `.bin` for each
- Dump the **ELF symbol map** with `arm-none-eabi-nm` and use it as ground truth
- Load each raw `.bin` into Binary Ninja at `0x10000000`
- **Break at** `main` on live silicon, even though `main` can move between programs
- **Hack each running target live** from Binary Ninja's Registers widget and Python console
- **Resolve the functions in the Binary Ninja GUI** using the ELF symbol map
- **Patch** the bytes that control behavior — strings, an IEEE-754 float, an immediate delay, and two `beq` targets — export, convert, and flash
- Understand how a **static** conditional is optimized away while a **dynamic** conditional must keep its `cmp` / `beq` / `bne` branches

How This Guide Works

Each project builds two files:

File	What it is	How we use it
`.elf`	The linked image with a full symbol table and DWARF	Ground truth for every function address and signature
`.bin`	The raw flash image, no headers, no symbols	The image we load into Binary Ninja and reverse

The `.bin` is built **from** the `.elf`, so the ELF tells you exactly what is at every address. We reverse-engineer the raw `.bin` the way a real extracted firmware image is reversed.

> **Build `Release`, not `Debug`.** Every address in this guide matches the current `Release` builds. `Release` folds the `static` helpers (`print_if_else`, `print_switch`, `sweep_servo`, `eval_if_else`, `process_servo_command`) into `main` and keeps the code layout stable. If you build `Debug`, the SDK function addresses move and the helpers stay separate, so nothing lines up. Always build `Release` for this lesson.

The order is **dynamic first, static second**, twice — once per project:

1. Break on the live target and prove what the code does.
2. Hack it live in the debugger and watch the behavior change.
3. Resolve the functions in Binary Ninja using the ELF symbol map.
4. Patch the bytes, export, convert, and flash.

Project	Prints	Also does	The hack
`0x001d_static-conditionals`	`1`, then `one`, forever	sweeps the SG90 servo 0° → 180°	angle `180` → `30`, delay `500` → `100`, string `1` → `2`, `one` → `fun`
`0x0020_dynamic-conditionals`	`1` + `one` or `2` + `two` from the keyboard	sweeps the servo on `1` / `2`	keys `1` / `2` → `x` / `y`, angle `180` → `30`, skip the prints to go stealth

> **Two conditionals, two fates.** In Project 1 the `choice` value is hard-coded (`int choice = 1;`), so the compiler proves the condition at compile time and **deletes the `cmp` and the dead branches** — that is a *static* conditional. In Project 2 `choice = getchar()`, so the value is only known at run time and the compiler must emit the `cmp` / `beq` / `bne` chain — that is a *dynamic* conditional. You will see both in the disassembly.

> **Addresses come from your build.** Every address here is from the `Release` build produced in Step 3 and was verified against the current `.elf` files with `arm-none-eabi-nm` and `arm-none-eabi-objdump`. Confirm against your own `.elf` with the command in Step 4.

> **The SVD file lives in `WEEK04`.** If you want the RP2350 peripheral register map for the PWM/UART side of the lab, it is `Embedded-Hacking/WEEK04/rp2350.svd` — it is **not** in `WEEK10`.

Background: PWM and the SG90 in one paragraph

A servo wants a **50 Hz** signal (a 20 ms frame). The RP2350 system clock is **150 MHz**; the `servo.c` driver divides that down to a **1 MHz** tick (1 tick = 1 µs) and wraps the counter at **20,000**, giving a 20 ms frame. The pulse width picks the angle: **1000 µs = 0°**, **1500 µs = 90°**, **2000 µs = 180°**. `servo_set_angle(float)` clamps the angle, maps it to a pulse in `[1000, 2000]`, and writes the PWM compare level. The float travels in a **general-purpose register (`r0`)**, not `s0` — you will see `vmov s14, r0` at the top of `servo_set_angle`. That is why the live angle hack edits `r0`.

Part 1: Build, Flash, and Get the Symbol Map

Step 1: Install the toolchain

Windows x64

- Install the **Raspberry Pi Pico** extension in VS Code. It installs the ARM GNU toolchain, CMake, Ninja, and the Pico SDK.
- Install **Binary Ninja Personal** and complete its license activation.
- Install **PuTTY** for the serial monitor.

macOS Apple Silicon

```
brew install cmake ninja
```

- Install **Binary Ninja Personal** and complete its license activation.
- Install the **Arm GNU Toolchain**, or let the VS Code Pico extension manage it.

Linux x64

```
sudo apt install cmake ninja-build gcc-arm-none-eabi libnewlib-arm-none-eabi git python3 openocd
minicom
```

- Install **Binary Ninja Personal** and complete its license activation.

Step 2: Verify your tools are the right architecture (do not skip this)

On **macOS Apple Silicon**, the most common failure is an Intel `x86_64` tool on your `PATH`:

```
zsh: bad CPU type in executable: cmake
```

You may have **two Homebrews**: the arm64 one at `/opt/homebrew` and the Intel one at `/usr/local`. If `/usr/local/bin` wins, every `brew` tool is x86_64. Check:

```
file "$(which cmake)"
file "$(which ninja)"
file "$(which arm-none-eabi-gdb)"
file "$(which arm-none-eabi-nm)"
file "$(which openocd)"
```

All must report `arm64`. If any is `x86_64`, put the Apple Silicon prefix first for the session and check again:

```
export PATH="/opt/homebrew/bin:$PATH"
hash -r
file "$(which cmake)"
```

To make it permanent, add that `export` to `~/.zshrc`. Do not use Rosetta as a fix; OpenOCD and GDB are exactly the kind of programs where a translation layer produces failures that look like debugger bugs.

Windows x64 and **Linux x64** do not have this problem. Skip to Step 3.

Step 3: Build the two projects with `Release`

Run this once inside `0x001d_static-conditionals/` and once inside `0x0020_dynamic-conditionals/`:

```
cmake -B build -G Ninja -DPICO_BOARD=pico2 -DPICO_PLATFORM=rp2350 -DCMAKE_BUILD_TYPE=Release
cmake --build build
```

Point Binary Ninja at this repository (once). Every console snippet below reads the repo root from `~/.embedded-hacking-repo`, so Binary Ninja never needs a database open and nothing is hardcoded. From the repo root, run once:

macOS / Linux:

```
pwd > ~/.embedded-hacking-repo
```

Windows (PowerShell):

```
(Get-Location).Path | Set-Content "$env:USERPROFILE\.embedded-hacking-repo"
```

Then build from the Binary Ninja console, so the whole build → patch → flash loop stays inside Binary Ninja. The console inherits a minimal `PATH` — on macOS just `/usr/bin:/bin:/usr/sbin:/sbin` — so it does not see Homebrew; add your package manager's `bin` first, then run plain `cmake`.

macOS Apple Silicon:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
os.environ["PATH"] = "/opt/homebrew/bin:" + os.environ["PATH"]  # the console's PATH omits Homebrew
for name in ("0x001d_static-conditionals", "0x0020_dynamic-conditionals"):
    proj = os.path.join(root, name)
    subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                    "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
    subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
for name in ("0x001d_static-conditionals", "0x0020_dynamic-conditionals"):
    proj = os.path.join(root, name)
    subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                    "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
    subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
for name in ("0x001d_static-conditionals", "0x0020_dynamic-conditionals"):
    proj = os.path.join(root, name)
    subprocess.run(["cmake", "-B", "build", "-G", "Ninja", "-DPICO_BOARD=pico2",
                    "-DPICO_PLATFORM=rp2350", "-DCMAKE_BUILD_TYPE=Release"], cwd=proj)
    subprocess.run(["cmake", "--build", "build"], cwd=proj)
```

Each build directory now contains the pair we need:

- `0x001d_static-conditionals/build/0x001d_static-conditionals.elf` and `.bin` — `.bin` is **8084** bytes (`0x1f94`)
- `0x0020_dynamic-conditionals/build/0x0020_dynamic-conditionals.elf` and `.bin` — `.bin` is **16188** bytes (`0x3f3c`)

If the ARM toolchain is not on your `PATH`, add `-DPICO_TOOLCHAIN_PATH=...`:

OS	Typical toolchain path
Windows x64	`C:/Program Files/Arm GNU Toolchain arm-none-eabi/14.2 rel1/bin`
macOS Apple Silicon	`/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin`
Linux x64	`/usr`

Step 4: Dump the ELF symbol map

This is the ground truth for the whole lesson. Run `arm-none-eabi-nm` on each ELF and keep the output in a terminal or a text file:

macOS Apple Silicon / Linux x64:

```
arm-none-eabi-nm -n --defined-only build/0x001d_static-conditionals.elf | grep -E ' [Tt] '
arm-none-eabi-nm -n --defined-only build/0x0020_dynamic-conditionals.elf | grep -E ' [Tt] '
```

Windows x64:

```
arm-none-eabi-nm -n --defined-only build\0x001d_static-conditionals.elf | Select-String ' [Tt] '
arm-none-eabi-nm -n --defined-only build\0x0020_dynamic-conditionals.elf | Select-String ' [Tt] '
```

Each line is `address type name`. The `T` / `t` type is a function. The signatures below come from the ELF's DWARF debug info queried with `arm-none-eabi-gdb -batch -ex "ptype <name>"`, so they are exact.

Project 1 — our code and the startup chain:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init`, `main`, `exit`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function (all three `static` helpers inlined)
`0x1000027c`	`servo_init`	`void servo_init(uint8_t)`	the `servo.c` init function
`0x10000318`	`servo_set_angle`	`void servo_set_angle(float)`	the `servo.c` angle setter (clamp + PWM level, all helpers inlined)

Project 1 — the GPIO, timer, stdio, and `puts` chain `main` reaches:

Address	ELF symbol	Signature	Role
`0x100003d8`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO function select
`0x10000e28`	`sleep_ms`	`void sleep_ms(uint32_t)`	SDK millisecond delay
`0x1000100c`	`time_us_64`	`uint64_t time_us_64(void)`	SDK microsecond clock

Address	ELF symbol	Signature	Role
`0x10001020`	`busy_wait_us`	`void busy_wait_us(uint64_t)`	UART timing loop
`0x100010a0`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`	SDK UART init
`0x10001274`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`	UART/servo clock lookup
`0x10001604`	`exit`	`void exit(int)`	C runtime exit
`0x1000160c`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x10001638`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`	CRLF output driver
`0x100016e8`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`	buffered string output
`0x100017d4`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`	enable the UART driver
`0x100017fc`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init
`0x1000188c`	`__wrap_puts`	`int __wrap_puts(const char*)`	the `puts` wrapper (both prints)
`0x10001a68`	`stdio_uart_init`	`void stdio_uart_init(void)`	SDK UART stdio init
`0x10001ba8`	`strlen`	`size_t strlen(const char*)`	C runtime string length

Project 2 — our code and the startup chain:

Address	ELF symbol	Signature	Role
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`	reset entry
`0x10000186`	`platform_entry`	`void platform_entry(void)`	calls `runtime_init`, `main`, `exit`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`	copies `.data` from flash to SRAM
`0x100001e4`	`_init`	`void _init(void)`	runs `.init_array`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`	C runtime boilerplate
`0x10000234`	`main`	`int main(void)`	the lesson function (both `static` helpers inlined)
`0x100002dc`	`servo_init`	`void servo_init(uint8_t)`	the `servo.c` init function
`0x10000378`	`servo_set_angle`	`void servo_set_angle(float)`	the `servo.c` angle setter (all helpers inlined)

Project 2 — the GPIO, timer, stdio, `printf`, `puts`, and `getchar` chain `main` reaches:

Address	ELF symbol	Signature	Role
`0x10000438`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`	SDK GPIO function select
`0x10000e88`	`sleep_ms`	`void sleep_ms(uint32_t)`	SDK millisecond delay
`0x1000106c`	`time_us_64`	`uint64_t time_us_64(void)`	SDK microsecond clock
`0x10001080`	`busy_wait_us`	`void busy_wait_us(uint64_t)`	UART timing loop
`0x10001100`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`	SDK UART init
`0x100012d4`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`	UART/servo clock lookup
`0x10002f90`	`vfctprintf`	`int vfctprintf(void (*)(char, void*), void*, const char*, va_list)`	printf format engine
`0x10002fec`	`exit`	`void exit(int)`	C runtime exit
`0x10002ff4`	`runtime_init`	`void runtime_init(void)`	SDK runtime init
`0x10003020`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`	CRLF output driver
`0x10003130`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`	buffered string output
`0x1000321c`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`	enable the UART driver
`0x10003244`	`stdio_init_all`	`bool stdio_init_all(void)`	SDK serial init
`0x10003250`	`__wrap_getchar`	`int __wrap_getchar(void)`	the `getchar` wrapper (reads the UART)
`0x10003344`	`__wrap_puts`	`int __wrap_puts(const char*)`	the `puts` wrapper
`0x10003380`	`__wrap_vprintf`	`int __wrap_vprintf(const char*, va_list)`	printf core
`0x10003444`	`__wrap_printf`	`int __wrap_printf(const char*, ...)`	the `printf` wrapper (`%s` calls)
`0x10003600`	`stdio_uart_init`	`void stdio_uart_init(void)`	SDK UART stdio init
`0x10003740`	`strlen`	`size_t strlen(const char*)`	C runtime string length

> `main` **is** `0x10000234` **in both projects.** In Project 1 the three `static` helpers are inlined into `main`; in Project 2 `eval_if_else` and `process_servo_command` are inlined, and `sweep_servo` with them. That is why both projects put `main` at the same address. In a `Debug` build the helpers stay separate and `main` moves — another reason to build `Release`.

> `printf` **in our source is** `__wrap_printf` **in the binary.** The SDK links our `printf` calls to its `__wrap_printf` wrapper, which forwards to `__wrap_vprintf`. Project 1 has no `printf` at all: the compiler replaced every `printf("...")` with a `__wrap_puts` because the strings have no format specifiers.

Step 5: Flash Project 1 and confirm `1` / `one` + servo sweep

A `.bin` has no headers, so OpenOCD must be told the base address `0x10000000`. From the repository root:

macOS Apple Silicon / Linux x64:

```
./flash.sh 0x001d_static-conditionals/build/0x001d_static-conditionals.bin
```

Windows x64 (PowerShell):

```
.\flash.ps1 -Bin 0x001d_static-conditionals\build\0x001d_static-conditionals.bin
```

Or flash from the Binary Ninja console:

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x001d_static-conditionals", "build", "0x001d_static-
conditionals.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x001d_static-conditionals", "build", "0x001d_static-
conditionals.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing in the background; log:", log)
```

Wait for `wrote 8084 bytes ...` and `** Verified OK **`. Open a serial monitor at **115200** baud:

- **Windows x64:** PuTTY → Connection type **Serial**, the Pico's COM port, speed `115200`.
- **macOS Apple Silicon:** `screen /dev/tty.usbmodem* 115200` (quit with `Ctrl-A` then `K`).
- **Linux x64:** `minicom -D /dev/ttyACM0 -b 115200`.

```
1
one
1
one
1
one
...
```

The servo sweeps **0°** → **180°** → **0°** once per second, and because `choice` is hard-coded the same two lines repeat forever.

Step 6: Flash Project 2 and confirm the dynamic behavior

```
# macOS / Linux
./flash.sh 0x0020_dynamic-conditionals/build/0x0020_dynamic-conditionals.bin
```

```
# Windows
.\flash.ps1 -Bin 0x0020_dynamic-conditionals\build\0x0020_dynamic-conditionals.bin
```

Or flash from the Binary Ninja console (same form, pointing at the Project 2 `.bin`):

macOS / Linux:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0020_dynamic-conditionals", "build", "0x0020_dynamic-
conditionals.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Windows:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
bin_path = os.path.join(root, "0x0020_dynamic-conditionals", "build", "0x0020_dynamic-
conditionals.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing in the background; log:", log)
```

Wait for `wrote 16188 bytes ...`. Nothing prints until you type. In the serial monitor:

- type `1` → the Pico prints `1` then `one`, and the servo sweeps **0°** → **180°**;
- type `2` → it prints `2` then `two`, and the servo sweeps **180°** → **0°**;
- type anything else → it prints `??` twice and waits for another key.

Part 2: Load the Raw `.bin` into Binary Ninja (Project 1)

Start from a fresh Binary Ninja state. If you already have a `.bndb` for this lesson, **close it and start over**; a stale database keeps old names and patches.

Step 7: Bring the raw `.bin` into Binary Ninja

A raw `.bin` has no headers, so Binary Ninja cannot know where it belongs or what architecture it is. You must supply both. If you just double-click the `.bin`, Binary Ninja may load it at address `0x0` with a guessed architecture, and every address in this lesson will be wrong.

1. Choose `File -> Open with Options...` (do **not** use plain `File -> Open`).
2. Select `0x001d_static-conditionals/build/0x001d_static-conditionals.bin`.
3. In the loader options, set:
    - **Architecture:** `thumb2` (the ARMv7-M / ARMv8-M Thumb-2 architecture, which covers the Cortex-M33)
    - **Platform:** `thumb2`
    - **Base Address:** `0x10000000` (the XIP flash base)
4. Click **Open**.

Binary Ninja analyzes the image and opens the linear view.

Verify the load before going further. Press `G`, type `0x10000000`, and read the first two words:

```
0x10000000   0x20082000   initial stack pointer
0x10000004   0x1000015d   reset vector (bit 0 = Thumb)
```

If you instead see data at `0x00000000`, or a vector word without bit 0 set, close the tab and repeat with `Open with Options`. The Cortex-M33 only executes Thumb-2, so `thumb2` is the only correct architecture.

> **Console equivalent:**
>
> ```
> load("0x001d_static-conditionals/build/0x001d_static-conditionals.bin",
>      options={"loader.imageBase": 0x10000000, "loader.platform": "thumb2"})
> ```

Step 8: Save it as a Binary Ninja database (`.bndb`)

Binary Ninja never writes back into the `.bin`. Your names, comments, types, and patches live in a separate `.bndb` database. Save one now, before you make any changes:

1. Choose `File -> Save As...`.
2. Save it next to the image as `0x001d_static-conditionals.bndb`.
3. From now on, save with `File -> Save` (`Cmd+S` on macOS, `Ctrl+S` on Windows/Linux) whenever you rename or patch.

File	Role
`0x001d_static-conditionals.bin`	the raw firmware image; Binary Ninja never modifies it
`0x001d_static-conditionals.bndb`	your analysis database: names, types, comments, and patches

When you come back later, **open the** `.bndb`, not the `.bin`; that restores all your work. You export the patched image out of this view later, in Step 19.

Step 9: The views you will use

- **Linear view:** the disassembly listing. You navigate, read, and patch here.
- **Graph view:** the control-flow graph of the current function.
- **Decompiler (HLIL):** the pseudo-C decompilation.
- **Hex view:** raw bytes, used for patching.
- **Function list:** the sidebar list of every detected function.

Navigation: `G` go to address, `N` rename, `Y` set type or signature, `;` add a comment. Breakpoints are set from the GUI through the GDB MI adapter — see Step 13.

> **macOS function keys:** the top-row `F` keys are usually mapped to system functions. Every step here uses menu paths that work without them.

Part 3: Dynamic — Break at `main` and Hack Live (Project 1)

Step 10: Start OpenOCD as a live debug server

Make sure no other OpenOCD is running; a forgotten server holds port `3333`.

macOS / Linux:

```
ps aux | grep -i openocd
```

Windows (PowerShell):

```
Get-Process | Where-Object { $_.ProcessName -like '*openocd*' }
```

Stop any leftover server gracefully:

macOS / Linux:

```
pkill -TERM -f openocd
```

Windows (PowerShell):

```
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

Start the server **parked at** `main`:

macOS Apple Silicon / Linux x64:

```
BP_ADDR=0x10000234 ./debug-server.sh
```

Windows x64 (PowerShell):

```
$env:BP_ADDR="0x10000234"; .\debug-server.ps1
```

Or start it from the Binary Ninja console, freeing the probe first and launching the server in the background so the console returns immediately:

macOS Apple Silicon / Linux x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen([os.path.join(root, "debug-server.sh")], cwd=root,
                     env=dict(os.environ, BP_ADDR="0x10000234"),
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

Windows x64:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step 3)
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop any running server first
log = os.path.join(root, "openocd.log")
p = subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                      os.path.join(root, "debug-server.ps1")], cwd=root,
                     env=dict(os.environ, BP_ADDR="0x10000234"),
                     stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("OpenOCD started (pid", p.pid, "); log:", log)
```

`Popen` returns in a few milliseconds; the server keeps running in the background. Check `openocd.log` for `Listening on port 3333`, then connect in Step 11.

Wait for:

```
Info : [rp2350.dap.core0] Examination succeed
Startup breakpoint at 0x10000234 (2-byte hardware execute, one-shot).
Info : starting gdb server for rp2350.dap.core0 on 3333
Info : Listening on port 3333 for gdb connections
```

> `BP_ADDR` **parks the core at** `main` **before any client connects.** The script arms a 2-byte hardware breakpoint and then does the startup `reset run`, so the core runs from the vector table and stops at your address with no debugger attached yet. When Binary Ninja connects a moment later, the first thing it reads is already the truth: `Stopped at 0x10000234`.

> **This startup stop is single-use.** OpenOCD flushes breakpoints when a client connects, so this one is gone once Binary Ninja attaches — fine for `main`, which only runs once per reset. Every breakpoint after that is set from the Binary Ninja GUI (Step 13) and is repeatable. To stop at `main` again, restart the server with `BP_ADDR` and reconnect.

> **Exactly one core.** The line must say `core0` and must **not** mention `core1`. Core1 is never started by this firmware; exposing it makes Binary Ninja read core1's reset-state registers, which are not real addresses, and OpenOCD floods the log with `Failed to read memory at 0xf0000000`. The scripts already use `USE_CORE=0`; do not change it.

> **Windows driver note:** the Debug Probe must use the **WinUSB** driver. If OpenOCD reports `unable to open CMSIS-DAP device`, install it with Zadig (select `Debug Probe (CMSIS-DAP)` → WinUSB).

Step 11: Connect Binary Ninja to the GDB server

1. Make sure the image is open and analyzed (Part 2) and the server from Step 10 is running (parked at `main`).
2. Choose `Debugger -> Connect to Remote Process`.
3. In the **adapter** dropdown, select **GDB MI**.
4. In the **connect** settings group, set **IP Address** to `127.0.0.1` and **Port** to `3333`.
5. Set **Full GDB Executable Path** to the `arm-none-eabi-gdb` from the **Arm GNU Toolchain 14.2.rel1**. It ships for all three hosts, and the Raspberry Pi Pico VS Code extension installs that same 14.2.rel1 toolchain (including `arm-none-eabi-gdb`) on all of them:

OS	`arm-none-eabi-gdb` path
macOS Apple Silicon	`/Applications/ArmGNUToolchain/14.2.rel1/arm-none-eabi/bin/arm-none-eabi-gdb` (or the Pico extension's `~/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb`)
Windows x64	`%USERPROFILE%\.pico-sdk\toolchain\14_2_Rel1\bin\arm-none-eabi-gdb.exe` (Pico extension), or `C:\Program Files (x86)\Arm GNU Toolchain arm-none-eabi\14.2 rel1\bin\arm-none-eabi-gdb.exe`
Linux x64	`~/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb` (Pico extension), or the `bin/` directory of the extracted `arm-gnu-toolchain-14.2.rel1-x86_64-arm-none-eabi` tarball

6. Click **Accept**.
7. Restrict the GDB memory map (prevents OpenOCD SWD faults and reconnect loops): In the bottom console panel, click the **Debugger Console** tab (or `View -> Tabs -> Debugger Console`) and enter:

```
source pico-gdb-memory.gdb
```

(If not running from the repository root, enter the absolute path to `pico-gdb-memory.gdb`*.)* Confirm `info mem` displays the Flash (`0x10000000 0x10400000 ro`) and SRAM (`0x20000000 0x20082000 rw`) ranges.

> **Why `source pico-gdb-memory.gdb` is required.** Binary Ninja's register widget automatically previews memory pointed to by registers. When a register contains an unmapped address (such as uninitialized data or constants like `0x69000000`), Binary Ninja probes it. Without a bounded memory map, OpenOCD attempts to read that invalid address over SWD, faults, and reconnects the debug port (`Info : SWD DPIDR 0x4c013477`). Sourcing `pico-gdb-memory.gdb` sets `inaccessible-by-default on` in GDB, causing GDB to intercept and reject invalid reads locally before any packet reaches OpenOCD, eliminating SWD reconnect storms.

> **Use the GDB MI adapter.** It launches a real `arm-none-eabi-gdb --interpreter=mi2` and lets Binary Ninja drive it, so breakpoints and stepping go through real GDB — which sends the correct 2-byte breakpoint length. Verified working end to end: connect, GUI breakpoints (**Add Hardware Breakpoint...**, hardware execute), **Step Into** / **Step Over**, and register edits.

> **Do NOT have any breakpoints set in Binary Ninja before you connect.** With the GDB MI adapter, attaching while Binary Ninja already has a breakpoint **hangs the session**. Start the server parked with `BP_ADDR` (Step 10), connect, and only add hardware breakpoints *after* the connection is up. This is a Binary Ninja bug; it is the single most common GDB MI failure.

> **The GDB executable path matters.** Use the **14.2.rel1** build on every OS (Windows, macOS, Linux). The 13.3.rel1 build did **not** connect in testing.
>
> **This step is temporary.** Vector35 plans to ship a GDB binary with the GDB MI adapter (Vector35/debugger#929, milestone *Langara*). Once that lands, Binary Ninja provides GDB itself and you will not need to set **Full GDB Executable Path** at all.

> **Do not pick Corellium.** Binary Ninja's adapter dropdown also lists **Corellium**, which is for Corellium's virtual devices and expects an API token, not a local OpenOCD server. Always read the label back and confirm it says **GDB MI** before clicking **Accept**.

> **The adapter and port are not saved in the `.bndb`.** Every time you relaunch Binary Ninja you must re-select **GDB MI**, re-enter port `3333`, and re-set the GDB path.

The target keeps running. Open the **Registers** tab (bug icon) and confirm you see live values. `pc` inside `0x10003xxx` and `sp` just below `0x20082000` are healthy.

> **If `pc` is `0x00000088`, `0x000000ec`, or `sp` is `0xf0000000`, the session is bad.** Restart the server, then restart Binary Ninja (a server restart while attached leaves Binary Ninja in a stale session), and connect again.

Step 12: Find `main` without relying on its address

`main` can move between programs, so we do not guess it. We follow the one fixed path to it. Press `G` and go to `0x10000000`:

```
0x10000000   0x20082000   initial stack pointer (top of SRAM)
0x10000004   0x1000015d   reset vector
```

Bit 0 of a vector is the Thumb bit, so `0x1000015d` means "start at `0x1000015c`". That is `_reset_handler`. Follow the reset path to `0x10000186`, `platform_entry`:

```
10000186 <platform_entry>:
10000186: ldr  r1, [pc, #80]
10000188: blx  r1
1000018a: ldr  r1, [pc, #80]
1000018c: blx  r1
1000018e: ldr  r1, [pc, #80]
10000190: blx  r1
10000192: bkpt 0x0000
```

The middle `blx` at `0x1000018c` is the call to `main`. `platform_entry` is byte-identical in both projects, so `0x1000018c` catches `main` no matter where the linker placed it. The literal pool at `0x100001dc` holds `main | 1`; clearing bit 0 gives `0x10000234`.

Step 13: Set a hardware breakpoint in the GUI

With the **GDB MI** adapter, Binary Ninja sets breakpoints through real GDB, which sends the correct 2-byte length, so you set them **in the UI**. There is no command port here.

Where you can stop

You want to stop at	Project 1 address	How	Repeatable?
`main`	`0x10000234`	The server starts parked there with `BP_ADDR=0x10000234` (Step 10), so Binary Ninja is already stopped at `main` when it connects.	No — `main` runs once per reset.
First `puts` (`"1\r\n"`)	`0x10000246`	Set a hardware breakpoint in the GUI, then click **Resume**.	Yes — fires on every iteration.
Second `puts` (`"one\r"`)	`0x1000024c`	Same.	Yes.
`servo_set_angle(0.0f)`	`0x10000252`	Same.	Yes.
`servo_set_angle(180.0f)`	`0x10000260`	Same — this is the angle we hack.	Yes.

Set a breakpoint in the GUI

1. Press `G`, type the address (for example `0x10000260`), and press Enter.
2. Set a **hardware execution** breakpoint at that address, either way:
   - `Debugger -> Add Hardware Breakpoint...` — a **hardware execute** (`HE`) breakpoint. **Use this one.**
   - click the line and press `F2` (`Debugger -> Toggle Breakpoint`) — a **software** breakpoint. It will **not** work here: the code is in read-only flash, so GDB cannot install it and the core just keeps running.
3. Click **Resume**. The core is already running the loop, so the breakpoint fires on the next iteration. Binary Ninja stops with the PC at the address and reports it as a **Breakpoint**.

> **No breakpoints before you connect.** With GDB MI, a breakpoint set before the connection hangs the session (Step 11). Start parked with `BP_ADDR`, connect, *then* add breakpoints.

> **Step Over on the raw `.bin` steps *into* calls.** The raw image has no symbol for `__wrap_puts` or `servo_set_angle`, so **Step Over** at a `bl` behaves like **Step Into**. When the lab needs to execute the call and then stop, it moves the breakpoint to the return site and clicks **Resume** instead (Step 14 shows this).

> **Never use Binary Ninja's Restart button.** On RP2350 it resets and halts inside the boot ROM (`pc=0x88`, `sp=0xf0000000`). To reset cleanly, restart the server with `BP_ADDR` and reconnect.

Step 14: HACK IT LIVE — change the servo angle from 180° to 30°

`main` loads the constant `0x43340000` (180.0f) into `r4` once, before the loop, then copies it into `r0` at `0x1000025e` right before the `servo_set_angle` call at `0x10000260`. We break on that call and change the angle live.

1. Press `G`, go to `0x10000260` (the `bl servo_set_angle` for 180.0°).
2. Set a **hardware execute** breakpoint there: `Debugger -> Add Hardware Breakpoint...`. (Do not use `F2` — that is a software breakpoint and will not work on read-only flash.)
3. Click **Resume** in Binary Ninja. The target is already running the loop, so the breakpoint fires on the next pass. Binary Ninja stops with the program counter at `0x10000260` and `r0 = 0x43340000` — the `mov r0, r4` at `0x1000025e` just loaded the 180.0f literal into `r0`.
4. Open the **Registers** widget (bug icon → **Registers**).
5. Find `r0`. Its value is `0x43340000`.
6. **Set `r0` to `0x41f00000` (30.0f).** From Binary Ninja's Python console (`Plugins -> Python Console`):

   ```
   dbg.set_reg_value("r0", 0x41f00000)  # 30.0f
   ```

   `dbg.set_reg_value(name, value)` writes one register (returns `True` on success). You can also right-click `r0` in the **Registers** widget, press `E` (edit), type `41f00000`, and press Enter. The widget may not repaint the value, but the write reaches the target.
7. **Move the breakpoint past the call.** Remove the breakpoint at `0x10000260` and set a hardware breakpoint at `0x10000264` (the `mov.w r0, #500` right after the call). Two reasons not to just click **Step Over**: a breakpoint left on the current PC re-traps the step, and Binary Ninja's **Step Over** steps *into* `servo_set_angle` on this raw `.bin`.
8. Click **Resume**. The core executes `bl servo_set_angle` with `r0 = 0x41f00000`, so this sweep ends at **30°** instead of 180°, then stops at `0x10000264`. Watch the servo arm.

> **`r4` is the real source — and it never reloads inside the loop.** `r4` is loaded once at `0x10000242` (before the loop starts at `0x10000244`) from the literal at `0x10000270`, so if you set `r4 = 0x41f00000` instead of `r0`, *every* pass uses 30° until the next reset. Editing `r0` changes only the current sweep because the next pass reloads `r0` from `r4`. Both edits are useful: `r0` shows a one-shot live change; `r4` shows a sticky one.

Step 14b: HACK THE STRING LIVE — change `one` to `fun`

The text `"one\r"` lives in flash (`.rodata`) at `0x10001c64`, and flash is **read-only at runtime** — a debugger write there does not stick. So you cannot overwrite the text in place. Instead you redirect the pointer: at the `puts` call, `r0` holds the string address, so you point `r0` at a replacement string you place in RAM.

1. Press `G`, go to `0x1000024c` (the second `bl __wrap_puts`) and set a **hardware execute** breakpoint. Resume; the loop hits it next pass. At the stop, `r0 = 0x10001c64` — the `ldr r0, [pc, #44]` at `0x1000024a` just loaded the `"one\r"` pointer from the literal at `0x10000278`.
2. Put the replacement string into free RAM at `0x20080000` from Binary Ninja's **Python console**:

   ```
   dbg.write_memory(0x20080000, b"fun\r\x00")  # puts appends the newline
   ```

   `dbg.write_memory(address, bytes)` is Binary Ninja's debugger memory-write API; it returns `True` on success. The bytes are `66 75 6e 0d 00` = `"fun\r\0"`. We keep the `\r` and let `puts` add the `\n`, exactly as the compiler does for the original `"one\r"`.
3. Point `r0` at that string:

   ```
   dbg.set_reg_value("r0", 0x20080000)
   ```

   (Or right-click `r0` in the **Registers** widget, press `E`, type `20080000`, and press Enter.)
4. Move the breakpoint past the call (remove it at `0x1000024c`, set one at `0x10000250`) and click **Resume**. The core runs `puts` with `r0` pointing at your RAM string, so this iteration prints:

   ```
   fun
   ```

   then stops at `0x10000250`.

Like the angle hack, this is **one iteration only**: the loop reloads `r0` from the literal pool on every pass, so the next line is `one` again. The permanent version is the static patch in Step 18.

Step 15: Why the hack reverts (and why we patch next)

Press **Resume**. The loop branches back to `0x10000244`, which reloads `r0` from `0x10001c64` and `0x1000025e` reloads `r0` from `r4`, so the next line is `one` and the next sweep ends at 180° again. The live edits changed one iteration only; nothing in RAM controls these values. To make the changes permanent we must patch the bytes — the static pass.

Press **Pause** to stop the output flood.

Step 15b: Kill the debugger and OpenOCD

The live hack is done. Do this **before** the static pass.

1. In the **Debugger** sidebar, click the **X** (**Kill**) (or `Debugger -> Kill`) to disconnect Binary Ninja.
2. **Kill does not stop the OpenOCD process** — `debug-server.sh` started it separately, and it keeps running and holding the probe. Stop it from the Binary Ninja console:

   macOS / Linux:

   ```
   import subprocess
   subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop the debug server, free the probe
   ```

   Windows:

   ```
   import subprocess
   subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop the debug server, free the
   probe
   ```

3. Confirm nothing is left: `pgrep -fl openocd` (macOS/Linux) prints nothing.

Part 4: Static — Resolve the Functions in Binary Ninja and Patch (Project 1)

Step 16: Resolve the functions in the Binary Ninja GUI

We now name the functions in Binary Ninja using the ELF symbol map from Step 4. Binary Ninja loaded the raw `.bin` with **no symbols**, so every function shows as `sub_<addr>` — resolving means giving each one its real name and signature.

Three keys do all the work:

Key	Binary Ninja action	Use it for
`G`	Go to address	Jump to a function's address
`Y`	**Change Type**	Set the function's signature. The dialog shows the full prototype, so this sets the name *and* the type in one step.
`N`	Rename	Rename only, when you just want the name and not the type

For each function below: `G` to its address, then `Y` **(Change Type)** and type the prototype from the table.

How to resolve a function in Binary Ninja (`Y`)

`Y` is the **Change Type** key, and it is what actually resolves the function — it turns `void sub_100017fc()` into `bool stdio_init_all(void)`. The Change Type dialog shows the full declaration (name and type), so typing the prototype sets both:

1. `G` to the function's address. The cursor lands on the function.
2. Press `Y`. In the Change Type dialog, type the prototype from the table exactly — for example `bool stdio_init_all(void)` — and press Enter.

If `Y` seems to do nothing, confirm the cursor is on the function, or right-click it and pick **Change Type...**. Binary Ninja parses what you type and silently keeps the old type if it does not parse, so glance at the header after each `Y`.

Worked example: `main`

1. Press `G`, type `0x10000234`, press Enter. The cursor lands on `sub_10000234`.
2. Press `Y` (Change Type), type `int main(void)`, press Enter.

> **Binary Ninja shows `int32_t` where Ghidra shows `int`.** After you set `int main(void)`, the decompiler header may read `int32_t main(void)`. That is the same type — on this platform `int` is 32 bits and Binary Ninja's parser normalises it to `int32_t`. Do not fight it; it is not an error.

Worked example: `servo_init`

1. `G` -> `0x1000027c`.
2. `Y` -> `void servo_init(uint8_t pin)`.

It takes a `uint8_t` pin number; `main` calls it with `6` (`movs r0, #6`).

Worked example: `servo_set_angle`

1. `G` -> `0x10000318`.
2. `Y` -> `void servo_set_angle(float degrees)`.

The float arrives in `r0` (soft-float ABI), not `s0`. The function opens with `vmov s14, r0` and then clamps the resulting pulse to `[1000, 2000]` — you can see `cmp.w r3, #2000` and `cmp.w r3, #1000` inside it.

Worked example: `__wrap_puts`

1. `G` -> `0x1000188c`.
2. `Y` -> `int __wrap_puts(const char *s)`.

Both prints in `main` land here. `printf("1\r\n")` has no format specifiers, so the compiler replaced it with `puts`; the `\n` was trimmed out of the string because `puts` adds one.

Worked example: `stdio_init_all`

1. `G` -> `0x100017fc`.
2. `Y` -> `bool stdio_init_all(void)`.

It returns `bool`, not `void` — the ELF says `_Bool stdio_init_all(void)`. Our `main` ignores the return value, so the decompiler still reads cleanly.

Worked example: `sleep_ms`

1. `G` -> `0x10000e28`.
2. `Y` -> `void sleep_ms(uint32_t ms)`.

Both delay instructions load `r0 = 0x1f4` (500) immediately before calling it.

The rest of the chain is the same two keystrokes per function (`G`, then `Y`). This is **our code plus the library functions it actually calls** — not the whole SDK.

The call chain for this project:

```
main
├── stdio_init_all ── stdio_uart_init ── gpio_set_function, uart_init, stdio_set_driver_enabled
│                                        └── uart_init ── clock_get_hz, busy_wait_us
├── servo_init ── gpio_set_function, clock_get_hz
├── __wrap_puts ── strlen, stdio_put_string ── time_us_64, strlen
└── servo_set_angle, sleep_ms
```

Project 1 — resolve every function in that chain:

Address	Rename to (`N`)	Signature (`Y`)
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`
`0x10000186`	`platform_entry`	`void platform_entry(void)`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`
`0x100001e4`	`_init`	`void _init(void)`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`
`0x10000234`	`main`	`int main(void)`
`0x1000027c`	`servo_init`	`void servo_init(uint8_t)`
`0x10000318`	`servo_set_angle`	`void servo_set_angle(float)`
`0x100003d8`	`gpio_set_function`	`void gpio_set_function(uint, gpio_function_t)`
`0x10000e28`	`sleep_ms`	`void sleep_ms(uint32_t)`
`0x1000100c`	`time_us_64`	`uint64_t time_us_64(void)`
`0x10001020`	`busy_wait_us`	`void busy_wait_us(uint64_t)`
`0x100010a0`	`uart_init`	`uint uart_init(uart_inst_t*, uint)`
`0x10001274`	`clock_get_hz`	`unsigned long clock_get_hz(clock_handle_t)`
`0x10001604`	`exit`	`void exit(int)`
`0x1000160c`	`runtime_init`	`void runtime_init(void)`
`0x10001638`	`stdio_out_chars_crlf`	`void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)`
`0x100016e8`	`stdio_put_string`	`int stdio_put_string(const char*, int, bool, bool)`
`0x100017d4`	`stdio_set_driver_enabled`	`void stdio_set_driver_enabled(stdio_driver_t*, bool)`
`0x100017fc`	`stdio_init_all`	`bool stdio_init_all(void)`
`0x1000188c`	`__wrap_puts`	`int __wrap_puts(const char*)`
`0x10001a68`	`stdio_uart_init`	`void stdio_uart_init(void)`
`0x10001ba8`	`strlen`	`size_t strlen(const char*)`

A `void` return type may not stick — here is the fix. Binary Ninja treats `void` as low-confidence, and its analysis can override it with an inferred type — most often `int32_t` on this 32-bit target. It is most visible on `_reset_handler` (a hand-written assembly entry that never returns normally), but it can happen to **any** function whose return type Binary Ninja thinks it can infer.

Setting the full signature with `Y` reproduces the unwanted `int32_t`, and `fn.return_type = ...` fails too. What works is the **return-value** setter:

```
from binaryninja import ReturnValue, Type
fn = bv.get_function_at(0x1000015c)
if fn is not None:
    fn.return_value = ReturnValue(Type.void())
```

That holds `_reset_handler` at `void` even after reanalysis. If it still will not stick, leave it — it does not affect the rest of the lesson.

Shortcut — resolves name *and* type for every function. Instead of doing `N` + `Y` by hand, paste this into Binary Ninja's Python console (`Plugins -> Python Console`). It sets each function's name and signature programmatically:

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
typedef unsigned long clock_handle_t;
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature)
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x1000027c: ("servo_init",                "void servo_init(uint8_t)"),
    0x10000318: ("servo_set_angle",           "void servo_set_angle(float)"),
    0x100003d8: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x10000e28: ("sleep_ms",                  "void sleep_ms(uint32_t)"),
    0x1000100c: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x10001020: ("busy_wait_us",              "void busy_wait_us(uint64_t)"),
    0x100010a0: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x10001274: ("clock_get_hz",              "unsigned long clock_get_hz(clock_handle_t)"),
    0x10001604: ("exit",                      "void exit(int)"),
    0x1000160c: ("runtime_init",              "void runtime_init(void)"),
    0x10001638: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x100016e8: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x100017d4: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x100017fc: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x1000188c: ("__wrap_puts",               "int __wrap_puts(const char*)"),
    0x10001a68: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x10001ba8: ("strlen",                    "size_t strlen(const char*)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    f = bv.get_function_at(addr)
    if f is not None:
        f.set_user_type(sig)
```

SDK type names (`stdio_driver_t` , `uart_inst_t` , `gpio_function_t` , plus `uint` , `va_list` , `clock_handle_t`) are **not** in the raw `.bin` . `set_user_type` re-parses each signature as C, so an undefined name raises `SyntaxError: unknown type name '...'` and stops the loop — it is not harmless. The `sdk` block above defines them first. Standard names (`uint8_t` , `uint32_t` , `uint64_t` , `bool` , `size_t`) are built in.

Step 17: Read `main` in the decompiler

Open the **Decompiler** view on `main`:

```
10000234 <main>:
10000234: push  {r3, r4, r5, lr}
10000236: bl    100017fc <stdio_init_all>
1000023a: movs  r0, #6
1000023c: bl    1000027c <servo_init>
10000240: movs  r5, #0
10000242: ldr   r4, [pc, #44]
10000244: ldr   r0, [pc, #44]
10000246: bl    1000188c <__wrap_puts>
1000024a: ldr   r0, [pc, #44]
1000024c: bl    1000188c <__wrap_puts>
10000250: mov   r0, r5
10000252: bl    10000318 <servo_set_angle>
10000256: mov.w r0, #500
1000025a: bl    10000e28 <sleep_ms>
1000025e: mov   r0, r4
10000260: bl    10000318 <servo_set_angle>
10000264: mov.w r0, #500
10000268: bl    10000e28 <sleep_ms>
1000026c: b.n   10000244 <main+0x10>
1000026e: nop
10000270: .word 0x43340000
10000274: .word 0x10001c5c
10000278: .word 0x10001c64
```

The whole program is one loop because the three `static` helpers were inlined:

- **No `cmp` anywhere.** `choice` is the constant `1`, so the compiler folded `if (choice == 1)` to always-true, deleted `else if (choice == 2)` and `else`, and left only the `1` and `one` prints. That is the static conditional.
- `r5 = 0` (`movs r5, #0` at `0x10000240`) is the `0.0f` angle; `r4` holds `0x43340000` (180.0f) from the literal pool at `0x10000270`.
- `0x10000274` and `0x10000278` point at `"1\r\n"` and `"one\r"` in `.rodata`.

The decompiler reads roughly:

```
int32_t main(void)
{
    stdio_init_all();
    servo_init(6);
    do
    {
        __wrap_puts("1\r\n");
        __wrap_puts("one\r");
        servo_set_angle(0.0f);
        sleep_ms(0x1f4);
        servo_set_angle(180.0f);
        sleep_ms(0x1f4);
    } while (true);
}
```

Step 18: Patch 1 — change the strings `1` to `2` and `one` to `fun`

The strings live in `.rodata`:

```
10000270: .word 0x43340000
10000274: .word 0x10001c5c
10000278: .word 0x10001c64
```

`0x10001c5c` holds `31 0d 00` = `"1\r"`, and `0x10001c64` holds `6f 6e 65 0d 00` = `"one\r"`. Change the first byte of each string in the **Hex** view (`View -> Hex`, lock off) or the Python console:

```
bv.write(0x10001c5c, b"\x32")  # "1" -> "2"
bv.write(0x10001c64, b"fun")  # "one" -> "fun"
print(bv.read(0x10001c5c, 4))  # -> b'2\r\x00\x00'
print(bv.read(0x10001c64, 6))  # -> b'fun\r\x00\x00'
```

Keep the replacement lengths identical: `"1"` is one byte, `"one"` is three. A shorter string must be padded and a longer one would run into the next string.

Step 18b: Patch 2 — change the angle from 180.0f to 30.0f

The 180.0f literal sits at `0x10000270`:

```
10000270: .word 0x43340000
```

It is loaded into `r4` at `0x10000242` and copied into `r0` before the second `servo_set_angle`. IEEE-754:

- `0x43340000` = 180.0f → little-endian bytes `00 00 34 43`
- `0x41f00000` = 30.0f → little-endian bytes `00 00 f0 41`

Option A — Hex view: go to `0x10000270` and change `00 00 34 43` to `00 00 f0 41`, then reanalyze.

Option B — Python console:

```
bv.write(0x10000270, bytes.fromhex("0000f041"))
print(bv.read(0x10000270, 4).hex(" "))  # -> 00 00 f0 41
```

`30.0 = 1.875 × 2^4`; sign `0`, exponent `127 + 4 = 131 = 0x83`, mantissa `0.875 = 0x700000` → `0x41f00000`.

Step 18c: Patch 3 — speed up the sweep from 500 ms to 100 ms

The compiler packed `500` directly into two 32-bit Thumb-2 `mov.w` instructions:

```
10000256: mov.w r0, #500
1000025a: bl    10000e28 <sleep_ms>
10000264: mov.w r0, #500
10000268: bl    10000e28 <sleep_ms>
```

Each `mov.w r0, #500` is the four bytes `4f f4 fa 70`. The four bytes for `mov.w r0, #100` are `4f f0 64 00` (verified by assembling `mov.w r0, #100` with `arm-none-eabi-as`). Change both:

```
for addr in (0x10000256, 0x10000264):
    bv.write(addr, bytes.fromhex("4ff06400"))  # mov.w r0, #100
```

> **Why the bytes change shape.** `500` does not fit in an 8-bit rotated immediate, so the encoder uses the `f4 4f`-family form `4f f4 fa 70`. `100` (`0x64`) does fit, so the encoder uses the `f04f` / `f0 4f` form `4f f0 64 00`. Same instruction, different immediate encoding. Both are exactly 4 bytes.

Verify all five patches:

```
for addr in (0x10001c5c, 0x10001c64, 0x10000270, 0x10000256, 0x10000264):
    print(hex(addr), bv.read(addr, 4).hex(" "))
# -> 0x10001c5c 32 0d 00 00
# -> 0x10001c64 66 75 6e 0d
# -> 0x10000270 00 00 f0 41
# -> 0x10000256 4f f0 64 00
# -> 0x10000264 4f f0 64 00
```

Step 19: Export the patched `.bin`

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size come from the view itself
out = os.path.join(os.path.join(root, "0x001d_static-conditionals", "build"), "0x001d_static-
conditionals-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 8084 /.../build/0x001d_static-conditionals-h.bin
```

Where the two numbers come from — nothing is hardcoded:

- `seg.start` is the image base Binary Ninja loaded the `.bin` at (`0x10000000`), the same value you pass to `uf2conv --base`.
- `seg.data_length` is the segment's size in the file (`0x1f94` = 8084). Exactly one segment carries data (the image); every peripheral and synthetic segment has `data_length == 0`, so `next(...)` picks the image.

> **No relative path.** Binary Ninja's Python console runs with a read-only working directory (inside the app bundle), so a relative `open(...)` fails with `OSError: [Errno 30] Read-only file system. root` (from `~/.embedded-hacking-repo`, Step 3) is the repo, so the file is written into the project's `build/`.

Step 20: Convert to UF2

Run from the project directory:

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x001d_static-conditionals-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x001d_static-conditionals-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

> **Or convert from the Binary Ninja console** — `chdir` to a writable directory first (the default one is read-only), then run the script:
>
> ```
> import os, sys, runpy
> os.chdir(os.path.join(root, "0x001d_static-conditionals", "build"))  # the project build dir
> (writable)
> sys.argv = ["uf2conv.py", "0x001d_static-conditionals-h.bin",
>             "--base", "0x10000000", "--family", "0xe48bff59", "--output", "hacked.uf2"]
> runpy.run_path("../../uf2conv.py", run_name="__main__")
> ```

Step 21: Flash and verify

Hold **BOOTSEL**, plug in the Pico 2, and drag `hacked.uf2` onto the `RP2350` drive. Or flash the `.bin` over the Debug Probe with SWD — no BOOTSEL — from the console (stop any running OpenOCD first, and use `Popen`, not `run`, so the console is not blocked):

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
bin_path = os.path.join(os.path.join(root, "0x001d_static-conditionals", "build"), "0x001d_static-
conditionals-h.bin")
log = os.path.join(os.path.join(root, "0x001d_static-conditionals", "build"), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
p = subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                     stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing in the background; log:", log)
```

Open the serial monitor:

```
2
fun
2
fun
2
fun
...
```

The servo now sweeps **0°** → **30°** and does it **5× faster**. **Five bytes changed, no source code.**

Part 5: Reflash Project 2 and Load It into Binary Ninja

Step 22: Reflash Project 2 and restart the session

Part 4 left the Pico running the patched Project 1 image. Put the original Project 2 back and start a fresh session.

1. Stop any running debug server so the flash script can use the probe:

```
# macOS / Linux
pkill -TERM -f openocd
```

```
# Windows
Get-Process openocd -ErrorAction SilentlyContinue | Stop-Process
```

2. Flash the original Project 2 image:

```
# macOS / Linux
./flash.sh 0x0020_dynamic-conditionals/build/0x0020_dynamic-conditionals.bin
```

```
# Windows
.\flash.ps1 -Bin 0x0020_dynamic-conditionals\build\0x0020_dynamic-conditionals.bin
```

Or do steps 1–2 from the Binary Ninja console:

macOS / Linux:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step
3)
bin_path = os.path.join(root, "0x0020_dynamic-conditionals", "build", "0x0020_dynamic-
conditionals.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # free the probe first
subprocess.Popen([os.path.join(root, "flash.sh"), bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
print("flashing Project 2 in the background; log:", log)
```

Windows:

```
import os, subprocess
root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()  # set once (Step
3)
bin_path = os.path.join(root, "0x0020_dynamic-conditionals", "build", "0x0020_dynamic-
conditionals.bin")
log = os.path.join(os.path.dirname(bin_path), "flash.log")
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # free the probe first
subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                  os.path.join(root, "flash.ps1"), "-Bin", bin_path],
                 stdout=open(log, "w"), stderr=subprocess.STDOUT)
print("flashing Project 2 in the background; log:", log)
```

3. Load Project 2 and save its database — see Step 22b.

Confirm the Pico responds to `1` / `2` again.

Step 22b: Load Project 2 into Binary Ninja and save the database

Exactly like Steps 7–8, but for Project 2. **Use** `File -> Open with Options...` (not plain `File -> Open`), select `0x0020_dynamic-conditionals/build/0x0020_dynamic-conditionals.bin`, and set:

- **Architecture:** `thumb2`
- **Platform:** `thumb2`
- **Base Address:** `0x10000000`

Click **Open**. Then press `G`, type `0x10000000`, and confirm the first two words:

```
0x10000000   0x20082000   initial stack pointer
0x10000004   0x1000015d   reset vector (bit 0 = Thumb)
```

If you see data at `0x00000000`, close the tab and redo it with `Open with Options`.

Save it with `File -> Save As...` as `0x0020_dynamic-conditionals.bndb` (next to the `.bin`). From now on open the `.bndb`, not the `.bin`; save with `Cmd+S` / `Ctrl+S` after every rename or patch.

> **Console equivalent:**
>
> ```
> load("0x0020_dynamic-conditionals/build/0x0020_dynamic-conditionals.bin",
>      options={"loader.imageBase": 0x10000000, "loader.platform": "thumb2"})
> ```

Part 6: Dynamic — Break at `main` and Hack Live (Project 2)

Step 23: Break at `main`

`main` is at `0x10000234` in this project too. Start the server parked at `main` (Step 10 form) and connect with the **GDB MI** adapter (Step 11):

1. Restart the server parked at `main`:

   macOS / Linux:

   ```
   BP_ADDR=0x10000234 ./debug-server.sh
   ```

   ```
   # Windows
   $env:BP_ADDR="0x10000234"; .\debug-server.ps1
   ```

   Or restart it from the Binary Ninja console:

   macOS / Linux:

   ```
   import os, subprocess
   root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
   log = os.path.join(root, "openocd.log")
   subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # kill any running server first
   subprocess.Popen([os.path.join(root, "debug-server.sh")], cwd=root,
                    env=dict(os.environ, BP_ADDR="0x10000234"),
                    stdout=open(log, "w"), stderr=subprocess.STDOUT, start_new_session=True)
   print("OpenOCD restarted parked at main; log:", log)
   ```

   Windows:

   ```
   import os, subprocess
   root = open(os.path.expanduser("~/.embedded-hacking-repo")).read().strip()
   log = os.path.join(root, "openocd.log")
   subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # kill any running server first
   subprocess.Popen(["powershell", "-ExecutionPolicy", "Bypass", "-File",
                     os.path.join(root, "debug-server.ps1")], cwd=root,
                    env=dict(os.environ, BP_ADDR="0x10000234"),
                    stdout=open(log, "w"), stderr=subprocess.STDOUT)
   print("OpenOCD restarted parked at main; log:", log)
   ```

2. Connect Binary Ninja (Step 11): adapter **GDB MI**, IP `127.0.0.1`, port `3333`.

The target is already halted at `main` when Binary Ninja connects, and the sidebar reads `Stopped at 0x10000234`.

Step 24: Read `main` and find the dynamic conditional

The whole loop is one function because both helpers were inlined. Notice the `cmp` / `beq` / `bne` chain the compiler had to keep this time:

```
10000234 <main>:
10000234: push   {r3, r4, r5, lr}
10000236: bl     10003244 <stdio_init_all>
1000023a: movs   r0, #6
1000023c: bl     100002dc <servo_init>
10000240: movs   r4, #0
10000242: ldr    r5, [pc, #124]
10000244: bl     10003250 <__wrap_getchar>
10000248: uxtb   r0, r0
1000024a: cmp    r0, #49
1000024c: beq.n  10000268 <main+0x34>
1000024e: cmp    r0, #50
10000250: beq.n  10000294 <main+0x60>
10000252: ldr    r0, [pc, #112]
10000254: bl     10003344 <__wrap_puts>
10000258: ldr    r0, [pc, #104]
1000025a: bl     10003344 <__wrap_puts>
1000025e: bl     10003250 <__wrap_getchar>
10000262: uxtb   r0, r0
10000264: cmp    r0, #49
10000266: bne.n  1000024e <main+0x1a>
10000268: ldr    r0, [pc, #92]
1000026a: bl     10003344 <__wrap_puts>
1000026e: ldr    r1, [pc, #92]
10000270: ldr    r0, [pc, #92]
10000272: bl     10003444 <__wrap_printf>
10000276: mov    r0, r4
10000278: bl     10000378 <servo_set_angle>
1000027c: mov.w  r0, #500
10000280: bl     10000e88 <sleep_ms>
10000284: mov    r0, r5
10000286: bl     10000378 <servo_set_angle>
1000028a: mov.w  r0, #500
1000028e: bl     10000e88 <sleep_ms>
10000292: b.n    10000244 <main+0x10>
10000294: ldr    r0, [pc, #60]
10000296: bl     10003344 <__wrap_puts>
1000029a: ldr    r1, [pc, #60]
1000029c: ldr    r0, [pc, #48]
1000029e: bl     10003444 <__wrap_printf>
100002a2: mov    r0, r5
100002a4: bl     10000378 <servo_set_angle>
100002a8: mov.w  r0, #500
100002ac: bl     10000e88 <sleep_ms>
100002b0: mov    r0, r4
100002b2: bl     10000378 <servo_set_angle>
100002b6: mov.w  r0, #500
100002ba: bl     10000e88 <sleep_ms>
100002be: b.n    10000244 <main+0x10>
100002c0: .word  0x43340000
100002c4: .word  0x1000381c
100002c8: .word  0x10003800
100002cc: .word  0x10003808
100002d0: .word  0x1000380c
100002d4: .word  0x10003814
100002d8: .word  0x10003818
```

Because `choice` is now `getchar()` and can be anything, the compiler cannot fold the condition. It emits the comparisons at `0x1000024a` (`cmp r0, #49` = `0x31` = `'1'`) and `0x1000024e` (`cmp r0, #50` = `0x32` = `'2'`), each followed by a `beq.n`. That is the dynamic conditional.

The string map, read straight from `.rodata`:

Literal	Points at	String
`0x100002c4`	`0x1000381c`	`"??\r"` (the `else` / `default` prints)
`0x100002c8`	`0x10003800`	`"1\r\n"` (the `'1'` print)
`0x100002cc`	`0x10003808`	`"one"` (the `printf` argument)
`0x100002d0`	`0x1000380c`	`"%s\r\n"` (the `printf` format)
`0x100002d4`	`0x10003814`	`"2\r\n"` (the `'2'` print)
`0x100002d8`	`0x10003818`	`"two"` (the `printf` argument)

> **Why the `else` path has two `puts` and a second `getchar`.** The compiler inlined both `eval_if_else` and `process_servo_command`; both have a "default" that prints `"??\r\n"`, and the shared string is emitted twice (`0x10000252` and `0x10000258`). The second `getchar` at `0x1000025e` is the compiler's rotated loop back-edge. The observable behavior is what matters: `1` → `1` + `one`, `2` → `2` + `two`, anything else → `??` twice.

Step 25: HACK IT LIVE — drive the branch with `r0`

`getchar` blocks until you press a key, so this breakpoint fires exactly when a key arrives. We stop right after the read and overwrite the value so the program takes whichever branch we want.

1. Press `G`, go to `0x1000024a` (the first `cmp r0, #49`). Set a **hardware execute** breakpoint: `Debugger -> Add Hardware Breakpoint...`.
2. Click **Resume** and type any key in the serial monitor — for example `a`. `getchar` returns, the `uxtb` at `0x10000248` runs, and the breakpoint fires at `0x1000024a` with `r0 = 0x61` (`'a'`).
3. **Set `r0` to `0x31` (`'1'`)** from the Python console:

   ```
   dbg.set_reg_value("r0", 0x31)  # force the '1' branch
   ```

   (Or right-click `r0` in the **Registers** widget, press `E`, type `31`, and press Enter.)
4. Remove the breakpoint at `0x1000024a` and click **Resume**. The core runs `cmp r0, #49`, sees the forced `0x31`, and takes the `'1'` branch — so even though you typed `a`, the Pico prints:

   ```
   1
   one
   ```

   and sweeps the servo **0° → 180°**.

> **Change the branch, not the register, if you prefer.** Setting `r0 = 0x32` instead forces the `'2'` path (`2`, `two`, servo 180° → 0°). Setting `r0` to anything else drops into the `??` path. One live register write steers the whole control-flow chain.

Step 25b: HACK THE STRING LIVE — change `1` to `7`

The `'1'` print uses the string at `0x10003800` (`"1\r\n"`). Exactly like Project 1, redirect `r0` to a RAM string at the `puts` call.

1. Press `G`, go to `0x10000268` (the `bl __wrap_puts` on the `'1'` path) and set a hardware execute breakpoint. Resume and type `1`. At the stop, `r0 = 0x10003800` — the `ldr r0, [pc, #92]` at `0x10000268` loaded the `"1\r\n"` pointer.
2. Write the replacement to RAM and repoint `r0`:

```
dbg.write_memory(0x20080000, b"7\r\x00")  # puts appends the newline
dbg.set_reg_value("r0", 0x20080000)
```

3. Remove the breakpoint at `0x10000268`, set one at `0x1000026a`, and click **Resume**. This iteration prints:

```
7
one
```

   One iteration only — the loop reloads `r0` from the literal pool each pass. The permanent version is the static patch in Step 27b.

Step 25c: Kill the debugger and OpenOCD

Same as Step 15b: click the **X** (**Kill**) in the **Debugger** sidebar (or `Debugger -> Kill`), then stop OpenOCD from the Binary Ninja console:

macOS / Linux:

```
import subprocess
subprocess.run(["pkill", "-TERM", "-f", "openocd"])  # stop the debug server, free the probe
```

Windows:

```
import subprocess
subprocess.run(["taskkill", "/F", "/IM", "openocd.exe"])  # stop the debug server, free the probe
```

Part 7: Static — Resolve the Functions and Patch (Project 2)

Step 26: Resolve the functions in the Binary Ninja GUI

Same two keys as Step 16 — `G` to the address, then `Y` (Change Type) to set the prototype — using the Project 2 ELF symbol map from Step 4.

Worked example: `main`

1. `G` -> `0x10000234`.
2. `Y` -> `int main(void)` (Binary Ninja shows `int32_t main(void)` — the same 32-bit `int`).

Worked example: `__wrap_getchar`

1. `G` -> `0x10003250`.
2. `Y` -> `int __wrap_getchar(void)`.

`getchar` returns an `int` in `r0`; `main` immediately narrows it with `uxtb r0, r0` before comparing.

Worked example: `__wrap_printf`

1. `G` -> `0x10003444`.
2. `Y` -> `int __wrap_printf(const char *fmt, ...)`. Keep the `...` — `printf` is variadic. It forwards to `__wrap_vprintf`.

Worked example: `__wrap_puts`

1. `G` -> `0x10003344`.
2. `Y` -> `int __wrap_puts(const char *s)`.

Worked example: `servo_set_angle`

1. `G` -> `0x10000378`.
2. `Y` -> `void servo_set_angle(float degrees)`.

The clamp constants are inside it:

```
100003ba: cmp.w   r3, #2000
100003be: it      cs
100003c0: movcs.w r3, #2000
100003c4: cmp.w   r3, #1000
100003c8: it      cc
100003ca: movcc.w r3, #1000
```

`0x7d0` is the 2000 µs maximum pulse and `0x3e8` is the 1000 µs minimum.

The call chain for this project:

```
main
├── stdio_init_all ── stdio_uart_init ── gpio_set_function, uart_init, stdio_set_driver_enabled
│                                        └── uart_init ── clock_get_hz, busy_wait_us
├── servo_init ── gpio_set_function, clock_get_hz
├── __wrap_getchar ── busy_wait_us
├── __wrap_puts ── strlen, stdio_put_string ── time_us_64, strlen
├── __wrap_printf ── __wrap_vprintf ── vfctprintf, stdio_out_chars_crlf, time_us_64
└── servo_set_angle, sleep_ms
```

Project 2 — resolve every function in that chain:

Address	Rename to (`N`)	Signature (`Y`)
`0x1000015c`	`_reset_handler`	`void _reset_handler(void)`
`0x10000186`	`platform_entry`	`void platform_entry(void)`
`0x1000019a`	`data_cpy`	`void data_cpy(void*, void*, void*)`
`0x100001e4`	`_init`	`void _init(void)`
`0x10000210`	`frame_dummy`	`void frame_dummy(void)`
`0x10000234`	`main`	`int main(void)`
`0x100002dc`	`servo_init`	`void servo_init(uint8_t)`
`0x10000378`	`servo_set_angle`	`void servo_set_angle(float)`

Address	Rename to (N)	Signature (Y)
0x10000438	gpio_set_function	void gpio_set_function(uint, gpio_function_t)
0x10000e88	sleep_ms	void sleep_ms(uint32_t)
0x1000106c	time_us_64	uint64_t time_us_64(void)
0x10001080	busy_wait_us	void busy_wait_us(uint64_t)
0x10001100	uart_init	uint uart_init(uart_inst_t*, uint)
0x100012d4	clock_get_hz	unsigned long clock_get_hz(clock_handle_t)
0x10002f90	vfctprintf	int vfctprintf(void (*)(char, void*), void*, const char*, va_list)
0x10002fec	exit	void exit(int)
0x10002ff4	runtime_init	void runtime_init(void)
0x10003020	stdio_out_chars_crlf	void stdio_out_chars_crlf(stdio_driver_t*, const char*, int)
0x10003130	stdio_put_string	int stdio_put_string(const char*, int, bool, bool)
0x1000321c	stdio_set_driver_enabled	void stdio_set_driver_enabled(stdio_driver_t*, bool)
0x10003244	stdio_init_all	bool stdio_init_all(void)
0x10003250	__wrap_getchar	int __wrap_getchar(void)
0x10003344	__wrap_puts	int __wrap_puts(const char*)
0x10003380	__wrap_vprintf	int __wrap_vprintf(const char*, va_list)
0x10003444	__wrap_printf	int __wrap_printf(const char*, ...)
0x10003600	stdio_uart_init	void stdio_uart_init(void)
0x10003740	strlen	size_t strlen(const char*)

Shortcut — resolves name *and* type for every function. Paste this into Binary Ninja's Python console:

```
from binaryninja import Symbol, SymbolType
# The raw .bin has no headers, so these SDK types don't exist. set_user_type()
# re-parses each signature as C, so an undefined name raises
# "SyntaxError: unknown type name '...'". Define them first.
sdk = bv.parse_types_from_string("""
typedef unsigned int uint;
typedef char* va_list;
typedef unsigned long clock_handle_t;
struct stdio_driver;
typedef struct stdio_driver stdio_driver_t;
struct uart_inst;
typedef struct uart_inst uart_inst_t;
enum gpio_function {
    GPIO_FUNC_XIP = 0, GPIO_FUNC_SPI = 1, GPIO_FUNC_UART = 2, GPIO_FUNC_I2C = 3,
    GPIO_FUNC_PWM = 4, GPIO_FUNC_SIO = 5, GPIO_FUNC_PIO0 = 6, GPIO_FUNC_PIO1 = 7,
    GPIO_FUNC_GPCK = 8, GPIO_FUNC_USB = 9, GPIO_FUNC_NULL = 0x1f,
};
typedef enum gpio_function gpio_function_t;
""")
for name, ty in sdk.types.items():
    bv.define_user_type(name, ty)

# address: (name, signature)
funcs = {
    0x1000015c: ("_reset_handler",            "void _reset_handler(void)"),
    0x10000186: ("platform_entry",            "void platform_entry(void)"),
    0x1000019a: ("data_cpy",                  "void data_cpy(void*, void*, void*)"),
    0x100001e4: ("_init",                     "void _init(void)"),
    0x10000210: ("frame_dummy",               "void frame_dummy(void)"),
    0x10000234: ("main",                      "int main(void)"),
    0x100002dc: ("servo_init",                "void servo_init(uint8_t)"),
    0x10000378: ("servo_set_angle",           "void servo_set_angle(float)"),
    0x10000438: ("gpio_set_function",         "void gpio_set_function(uint, gpio_function_t)"),
    0x10000e88: ("sleep_ms",                  "void sleep_ms(uint32_t)"),
    0x1000106c: ("time_us_64",                "uint64_t time_us_64(void)"),
    0x10001080: ("busy_wait_us",              "void busy_wait_us(uint64_t)"),
    0x10001100: ("uart_init",                 "uint uart_init(uart_inst_t*, uint)"),
    0x100012d4: ("clock_get_hz",              "unsigned long clock_get_hz(clock_handle_t)"),
    0x10002f90: ("vfctprintf",                "int vfctprintf(void (*)(char, void*), void*, const
char*, va_list)"),
    0x10002fec: ("exit",                      "void exit(int)"),
    0x10002ff4: ("runtime_init",              "void runtime_init(void)"),
    0x10003020: ("stdio_out_chars_crlf",      "void stdio_out_chars_crlf(stdio_driver_t*, const
char*, int)"),
    0x10003130: ("stdio_put_string",          "int stdio_put_string(const char*, int, bool,
bool)"),
    0x1000321c: ("stdio_set_driver_enabled",  "void stdio_set_driver_enabled(stdio_driver_t*,
bool)"),
    0x10003244: ("stdio_init_all",            "bool stdio_init_all(void)"),
    0x10003250: ("__wrap_getchar",            "int __wrap_getchar(void)"),
    0x10003344: ("__wrap_puts",               "int __wrap_puts(const char*)"),
    0x10003380: ("__wrap_vprintf",            "int __wrap_vprintf(const char*, va_list)"),
    0x10003444: ("__wrap_printf",             "int __wrap_printf(const char*, ...)"),
    0x10003600: ("stdio_uart_init",           "void stdio_uart_init(void)"),
    0x10003740: ("strlen",                    "size_t strlen(const char*)"),
}
for addr, (name, sig) in funcs.items():
    bv.define_user_symbol(Symbol(SymbolType.FunctionSymbol, addr, name))
    f = bv.get_function_at(addr)
    if f is not None:
        f.set_user_type(sig)
```

Step 27: Patch 1 — change the servo angle from 180.0f to 30.0f

The 180.0f literal sits at `0x100002c0`, loaded into `r5` at `0x10000242`:

```
100002c0: .word 0x43340000
```

Change the four bytes exactly as in Project 1:

```
bv.write(0x100002c0, bytes.fromhex("0000f041"))  # 180.0f -> 30.0f
print(bv.read(0x100002c0, 4).hex(" "))  # -> 00 00 f0 41
```

Both servo paths (`0x10000284` / `0x100002a2` use `r5` , `0x10000276` / `0x100002b0` use `r4` = 0.0f) now cap at 30°.

Step 27b: Patch 2 — rewrite the secret keys `1` → `x` and `2` → `y`

The two key comparisons are at `0x1000024a` and `0x1000024e` :

```
1000024a: cmp r0, #49
1000024e: cmp r0, #50
```

The immediate is the low byte of the 16-bit `cmp` encoding: `0x31` at `0x1000024a` and `0x32` at `0x1000024e` . Change them to `x` (`0x78`) and `y` (`0x79`):

```
bv.write(0x1000024a, b"\x78")  # cmp r0, #0x78  ('x')
bv.write(0x1000024e, b"\x79")  # cmp r0, #0x79  ('y')
print(bv.read(0x1000024a, 2).hex(" "))  # -> 78 28
print(bv.read(0x1000024e, 2).hex(" "))  # -> 79 28
```

Now `x` takes the old `1` path and `y` takes the old `2` path.

Step 27c (optional): Patch 3 — make `x` and `y` stealth (skip the prints)

The original `1` / `2` paths print before moving the servo. To make the new `x` / `y` keys silent, redirect the two `beq.n` targets straight to the servo code, skipping both prints. The current targets are `0x10000268` (the `1` print block) and `0x10000294` (the `2` print block); the servo code starts at `0x10000276` (`mov r0, r4`) and `0x100002a2` (`mov r0, r5`).

Address	Instruction	Before	After	New target
`0x1000024c`	`beq.n`	`0c d0`	`13 d0`	`0x10000276` (skip `1` / `one` prints)
`0x10000250`	`beq.n`	`20 d0`	`27 d0`	`0x100002a2` (skip `2` / `two` prints)

```
bv.write(0x1000024c, bytes.fromhex("13d0"))  # beq.n -> 0x10000276
bv.write(0x10000250, bytes.fromhex("27d0"))  # beq.n -> 0x100002a2
```

> **How the encoding was chosen.** A 16-bit conditional branch is `1101 cond imm8` ; the target is `PC + 4 + (imm8 << 1)` . For `0x1000024c` → `0x10000276` : `(0x276 - 0x250) / 2 = 0x13` . For `0x10000250` → `0x100002a2` : `(0x2a2 - 0x254) / 2 = 0x27` . Both were verified by patching a copy of the raw `.bin` and disassembling it with `arm-none-eabi-objdump` . This patch is optional; the key rewrite in Step 27b works without it (it just still prints).

Step 28: Export, convert, and flash

```
import os
seg = next(s for s in bv.segments if s.data_length)  # the loadable image segment
data = bv.read(seg.start, seg.data_length)  # base + size from the view itself
out = os.path.join(os.path.join(root, "0x0020_dynamic-conditionals", "build"), "0x0020_dynamic-
conditionals-h.bin")
open(out, "wb").write(data)
print(len(data), out)  # -> 16188 /.../build/0x0020_dynamic-conditionals-h.bin
```

`seg.data_length` is the image size (`0x3f3c` = 16188) read from the view — nothing hardcoded.

macOS Apple Silicon / Linux x64:

```
python3 ../uf2conv.py 0x0020_dynamic-conditionals-h.bin \
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Windows x64:

```
python ..\uf2conv.py 0x0020_dynamic-conditionals-h.bin ^
  --base 0x10000000 --family 0xe48bff59 --output hacked.uf2
```

Or run the conversion from the Binary Ninja console, exactly as in Step 20 (`os.chdir` to the build dir, then `runpy.run_path("../../uf2conv.py", run_name="__main__")` with `sys.argv` set to the arguments above).

Hold **BOOTSEL**, plug in the Pico 2, drag `hacked.uf2` onto the `RP2350` drive. Or flash the `.bin` over the Debug Probe with SWD — no BOOTSEL — from the console, exactly as in Step 21.

Step 29: Verify

Open the serial monitor:

- type `x` → with Step 27c applied, **no output** and the servo sweeps silently; without it, `x` prints `x` … actually it prints the original strings `1` / `one` because only the comparison changed (the print strings are untouched). With Step 27c applied the prints are skipped entirely — a **stealth command**.
- type `y` → likewise silent (Step 27c), and the servo sweeps the other way.
- the servo's maximum angle is now **30°**, not 180°.
- the original `1` and `2` keys no longer match the comparisons.

We changed the servo angle and hid two secret keys, with a handful of bytes and no source code.

Cheatsheet

Binary Ninja GUI actions

Action	How
Go to address	`G`
Rename function/symbol	`N`

Action	How
Set type or signature	`Y`
Add comment	`;`
Open Hex view	`View -> Hex`
Enable hex editing	Toggle the lock in the status bar
Reanalyze after a patch	Right-click function -> `Reanalyze`
Edit a register live	`dbg.set_reg_value("r0", 0x41f00000)` in the Python console (or right-click the register, press `E`, type hex, Enter)
Write debugger memory	`dbg.write_memory(0x20080000, b"fun\r\x00")`
Set a breakpoint	`Debugger -> Add Hardware Breakpoint...` (hardware execute). Do **not** use `F2` — software breakpoints cannot be written to read-only flash.
Move a breakpoint	Remove it and set it at the new address in the GUI
Confirm what is armed	The **Breakpoints** widget lists it
Apply the ELF symbol map	Paste the Python snippet from Step 16 / 26 into the Python Console

OpenOCD server and reset

The server runs with `gdb_breakpoint_override hard` so that flash-writes are never attempted. Breakpoints in this lab are set in the Binary Ninja GUI through the **GDB MI** adapter (Step 13).

Action	Command
Start the server parked at `main`	macOS/Linux: `BP_ADDR=0x10000234 ./debug-server.sh` — Windows: `$env:BP_ADDR="0x10000234"; .\debug-server.ps1` (one-shot)
Break on a loop address in a running target	set a hardware breakpoint in the GUI, then **Resume** — repeatable
Kill the debugger	click **X** in the **Debugger** sidebar, or `Debugger -> Kill`
Stop OpenOCD	macOS/Linux: `pkill -TERM -f openocd` — Windows: `taskkill /F /IM openocd.exe`
Make Binary Ninja stepping work	`rp2350.dap.core0 configure -rtos none` (already in the scripts)
Step without re-trapping	move the breakpoint off the current PC first, then **Step Into**/**Step Over**
Reset without desyncing Binary Ninja	**Detach**, `reset run`, reconnect — never `reset run` while attached

Where you can stop

Stop at	Project 1 `0x001d`	Project 2 `0x0020`
`main` (once per reset)	`0x10000234`	`0x10000234`
First `puts`	`0x10000246` (`"1\r\n"`)	`0x10000268` (`"1\r\n"`)
Second `puts`	`0x1000024c` (`"one\r"`)	—
`getchar` return (dynamic key)	—	`0x1000024a` (`cmp r0, #0x31`)
`servo_set_angle(0.0f)`	`0x10000252`	`0x10000276` / `0x100002a2`
`servo_set_angle(180.0f)`	`0x10000260`	`0x10000284` / `0x100002b0`

Every address and byte we changed

Project	Address	Before	After	Effect
`0x001d`	`0x10001c5c`	`31`	`32`	prints `2` instead of `1`
`0x001d`	`0x10001c64`	`6f 6e 65`	`66 75 6e`	prints `fun` instead of `one`
`0x001d`	`0x10000270`	`00 00 34 43`	`00 00 f0 41`	servo max angle `180.0f` → `30.0f`
`0x001d`	`0x10000256`	`4f f4 fa 70`	`4f f0 64 00`	first `sleep_ms` `500` → `100`
`0x001d`	`0x10000264`	`4f f4 fa 70`	`4f f0 64 00`	second `sleep_ms` `500` → `100`
`0x0020`	`0x1000024a`	`31`	`78`	compare `'1'` → `'x'`
`0x0020`	`0x1000024e`	`32`	`79`	compare `'2'` → `'y'`
`0x0020`	`0x100002c0`	`00 00 34 43`	`00 00 f0 41`	servo max angle `180.0f` → `30.0f`
`0x0020`	`0x1000024c`	`0c`	`13`	`beq.n` skips the `1` / `one` prints (optional)
`0x0020`	`0x10000250`	`20`	`27`	`beq.n` skips the `2` / `two` prints (optional)

Raw image facts

Item	Value
Build type	`Release`
Load base address	`0x10000000`
Project 1 size	`8084` bytes
Project 2 size	`16188` bytes
Initial stack pointer (both)	`0x20082000`
Reset vector (both)	`0x1000015d`
Fixed `main` anchor (both)	`0x1000018c`
`main` (both)	`0x10000234`

Item	Value
`servo_set_angle`, Project 1	`0x10000318`
`servo_set_angle`, Project 2	`0x10000378`
Project 1 `180.0f` literal	`0x10000270`
Project 2 `180.0f` literal	`0x100002c0`
Project 1 `one` string	`0x10001c64`
Project 2 `one` string	`0x10003808`
RP2350 UF2 family ID	`0xe48bff59`

Troubleshooting

Binary Ninja hangs or crashes when you connect (macOS 27)

Three different causes have been seen on this setup; check them in this order.

- **A breakpoint set before connecting.** With the **GDB MI** adapter, if the binary view already has a breakpoint, the session hangs. Start parked with `BP_ADDR`, connect, then add breakpoints.
- **The wrong GDB executable.** Point **Full GDB Executable Path** at the **14.2.rel1** toolchain (Step 11). The 13.3.rel1 build did not connect in testing.
- **The LLDB adapter.** A crash report with `libdebuggercore.dylib -> std::terminate() -> abort()` and `liblldb` in the stack is the **LLDB** adapter, not GDB MI. Avoid LLDB on this setup.

If Binary Ninja hangs, force-quit it; the connect dialog has no working Cancel. The static steps (resolve, patch, export, flash) never touch the debugger and always work.

GDB MI hangs when you connect (a breakpoint already existed)

With the **GDB MI** adapter, if Binary Ninja already has a breakpoint set when you connect, the session **hangs**. The working order is:

1. Start the server parked, e.g. `BP_ADDR=0x10000234 ./debug-server.sh` (Windows: `$env:BP_ADDR="0x10000234"; .\debug-server.ps1`).
2. Connect with the **GDB MI** adapter.
3. Only *then* set hardware breakpoints in the UI.

Step Into / Step Over does nothing (PC never moves)

Two causes have been seen on this target.

1. **A breakpoint on the current PC re-traps the step.** OpenOCD's step-over-breakpoint logic fails with `Duplicate Breakpoint address` and the PC stays put. Fix: move the breakpoint off the current PC (in the GUI), then step.
2. **The `hwthread` RTOS (GDB RSP adapter only).** With the **GDB RSP** adapter, OpenOCD can log `fake step thread 0` and reply without stepping. Fix: `rp2350.dap.core0 configure -rtos none`. **GDB MI does not hit this.**

`zsh: bad CPU type in executable: cmake`

An Intel `x86_64` tool is on your `PATH` on Apple Silicon. Run Step 2: `export PATH="/opt/homebrew/bin:$PATH"`, then `hash -r`.

My addresses do not match this guide

You probably built `Debug`. This lesson is a `Release` build. Re-run Step 3 with `-DCMAKE_BUILD_TYPE=Release`. A `Debug` build moves the SDK functions and keeps the `static` helpers separate.

A breakpoint never fires

First, confirm you actually set one, and that it is a **hardware** breakpoint. `Debugger -> Add Hardware Breakpoint...` (hardware execute) should land in the **Breakpoints** widget. If nothing lands, or the core keeps running, you probably used `F2` (`Toggle Breakpoint`) — a software breakpoint cannot be written to read-only flash.

Then check the order and the state:

- **Arm it only after Binary Ninja is connected.** OpenOCD flushes every breakpoint when a client attaches, so anything armed earlier is gone. This also applies to `BP_ADDR` on the startup command line.
- **Is the core running?** If it is stopped, click **Resume**.
- **Does the address get reached again?** `main` runs once per reset, so use `BP_ADDR` at startup rather than `reset run` while attached. Loop addresses such as `0x10000260` fire on the next pass with no reset. For Project 2's `0x1000024a` you must also press a key, because it sits right after the blocking `getchar`.

I edit `r0` (or another register) and it reverts

For the Project 1 angle, `mov r0, r4` at `0x1000025e` reloads `r0` on every pass, so the edit is visible for one sweep unless you stop the core again. Editing `r4` instead makes it stick, because `r4` is loaded once before the loop. For Project 2, `getchar` reloads `r0` on every key press. The edit sticks only while the core is **genuinely stopped** at the breakpoint.

> **The Registers widget is a snapshot, not a live view.** Binary Ninja reads the registers at each stop and shows that snapshot; it does not poll the target. A value changed outside Binary Ninja will not appear until the next stop.

The string hack does nothing (or prints garbage)

Pick a RAM address that is free — `0x20080000` is safe here (well above the `.data` / `.bss` end at about `0x2000062c`). Write a NUL-terminated string, and remember `__wrap_puts` appends its own `\n`, so keep the `\r` but not the `\n` (write `b"fun\r\x00"`). Then set `r0`, not `r1`.

The patched string shifted `printf` output

In Project 1 both prints are `puts`, so only whole-string replacement matters. Keep `"one"` → `"fun"` exactly three bytes; a longer string would overwrite the `\r` terminator and a shorter one would leave a stray character.

`mov.w r0, #100` corrupts the instruction

Use the four bytes `4f f0 64 00`, not the first two bytes of the `500` encoding. `500` needs the `4f f4 …` form; `100` uses the `4f f0 …` form. Both are 4 bytes. Verify with `arm-none-eabi-objdump` (Step 20/21 context) or by re-reading the bytes in Binary Ninja.

Project 2's `x` / `y` still print

Step 27b only changes the *comparison* values. To make the keys silent you must also apply the optional `beq` redirects in Step 27c. If the servo moves but the terminal still shows `1` / `one`, you applied 27b but not 27c.

The optional `beq` redirect sends execution somewhere wrong

Recompute from the ELF: `target = PC + 4 + (imm8 << 1)`. For `0x1000024c` the servo code is at `0x10000276` (imm8 `0x13`); for `0x10000250` it is at `0x100002a2` (imm8 `0x27`). Confirm the byte pair you write is little-endian (`13 d0`, `27 d0`).

The serial capture is garbage on macOS

Reading `/dev/cu.usbmodem*` with a bare `read()` returns garbage. Set **raw termios at 115200** first, or just use `screen /dev/cu.usbmodem* 115200`, which does it for you.

It worked for a second, then stopped (Binary Ninja's view desyncs)

The main cause is **driving the core from the OpenOCD command port while Binary Ninja is connected**. If you must reset, **Detach first**, reset, then reconnect. Never leave a breakpoint on the PC you are about to step or resume from.

The console floods with `Failed to read memory` or `SWD DPIDR` reconnects

Two causes:

1. **Core 1 is exposed.** The scripts must run with `USE_CORE=0`. Stop the server, confirm only `core0` is reported, restart, then restart Binary Ninja.
2. **Binary Ninja probed an unmapped address for a register preview.** When a register holds an arbitrary integer or uninitialized value (e.g. `0x69000000`), Binary Ninja tries to preview that memory address, triggering an OpenOCD SWD fault and debug-port reconnect (`SWD DPIDR 0x4c013477`). In the **Debugger Console**, run `source pico-gdb-memory.gdb`. This restricts GDB to valid Flash and SRAM, filtering invalid reads locally so OpenOCD's SWD connection stays completely stable.

The decompiler still shows the old value after patching

Right-click the function and choose `Reanalyze`.

Fallback: do the dynamic steps with GDB (macOS 27)

If Binary Ninja's debugger crashes on attach on macOS 27, you can still do the live hacks with the ARM GDB from the toolchain, against the same OpenOCD server. The addresses and register values are identical to the GUI steps.

Start the debug server (Step 10), then in a new terminal:

```
arm-none-eabi-gdb
```

At the `(gdb)` prompt:

```
set architecture armv8-m.main
target extended-remote :3333
hbreak *0x10000260
continue
```

Do **not** run `monitor reset run` before `hbreak`. `0x10000260` is inside `main`'s loop, so the breakpoint fires on the next iteration with no reset. GDB stops at the `servo_set_angle` call:

```
info registers pc r0  # pc = 0x10000260, r0 = 0x43340000
set $r0 = 0x41f00000
continue
```

The servo's next sweep ends at 30° — the same temporary live hack as editing `r0` in the Binary Ninja Registers widget. For the string hack, break at `0x1000024c`, then `set {char[5]}0x20080000 = "fun\r"` and `set $r0 = 0x20080000`.

Project 2 is the same with the other call site and value:

```
hbreak *0x1000024a
continue
info registers pc r0  # r0 holds the key you typed
set $r0 = 0x31
continue
```

`hbreak` sets a hardware breakpoint, which is required for read-only flash. It works from plain GDB because GDB sends the 2-byte length the Cortex-M33 comparators need. Binary Ninja's **GDB MI** adapter goes through the same GDB, so its GUI breakpoints work too.

Glossary

Term	Definition
`beq`	Branch if Equal — ARM conditional jump, taken when the Z flag is set
`bne`	Branch if Not Equal — ARM conditional jump, taken when the Z flag is clear
`.bss`	Section for uninitialized global variables; zeroed by startup code
`.data`	Section for initialized global variables; copied from flash to SRAM at boot
Dynamic conditional	A condition whose value is only known at run time (e.g. `getchar()`), so the compiler must emit the comparisons and branches
`.elf`	Linked image with the symbol table; the ground truth for addresses and names
GPIO	General Purpose Input/Output — controllable pins on the microcontroller
Hardware breakpoint	A breakpoint serviced by the CPU comparators, required for read-only flash
Inlining	The optimizer replacing a function call with the function body; why the `static` helpers disappear from `main` in `Release`
Literal pool	A block of 32-bit constants that Thumb-2 code reaches with PC-relative `ldr`

Term	Definition
PWM	Pulse Width Modulation — a variable pulse-width signal; 50 Hz for the SG90
`.rodata`	Read-only section for constants and string literals; stays in flash
SG90	A common 0°–180° hobby servo driven by a 1–2 ms pulse every 20 ms
SIO	Single-cycle I/O — the fast GPIO block in the RP2350, at `0xd0000000`
Static conditional	A condition whose value is known at compile time, so the compiler folds it and deletes dead branches
Thumb bit	Bit 0 of a Cortex-M function pointer; selects Thumb instruction mode
UF2	USB Flashing Format — the file format the Pico 2 bootloader accepts
Vector table	The first words of flash: initial stack pointer and exception vectors

---

Remember: the ELF tells you what every address is, and the `.bin` is what you actually patch. Prove the behavior dynamically, resolve the names from the ELF, then patch the bytes and flash.